Assignment no:- 1

Name: Yash Gawade | Roll No. 46 | PRN no:1252090031

1.Perform the following operations using Python on suitable data sets:

Read data from different formats (like csv, xls) Find Shape of Data Find Missing Values Find data type of each column Finding out Zero's Indexing and selecting data, sort data, Describe attributes of data, checking data types of each column, counting unique values of data, format of each column, converting variable data type (e.g. from long to short, vice versa) Important libraries which required with dealing the data

```
import pandas as pd
import numpy as np
```

Student Data set

```
import pandas as pd;
data = {
"Student ID": [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
"Name": ["Shantanu", "Sai", "Jay", "Snehal", "Rahul", "Pooja", "Kunal", "Anjali", "Saurabh", "Neha"],
"Age": [18, 15, 28, 12, 20, 18, 19, 20, 19, 18],
"Gender": ["Male", "Male", "Male", "Female", "Male", "Female", "Male", "Female", "Male", "Female"],
"Department": ["CSE AI", "CSE", "CSE AI", "CSE", "IT", "CSE AI", "IT", "CSE", "CSE AI", "IT"],
"Marks": [9.1, 8.6, 8.9, 9.3, 8.2, 9.0, 8.4, 9.2, 8.7, 9.1],
"Attendance": [95, 94, 93, 92, 92, 98, 90, 91, 93, 95],
"Internship": ["Yes", "Yes", "Yes", "Yes", "No", "Yes", "No", "Yes", "Yes", "Yes"]
}
df = pd.DataFrame(data)
print(df)
```

```
   Student ID      Name  Age  Gender Department  Marks  Attendance Internship
0         101  Shantanu   18    Male     CSE AI    9.1          95        Yes
1         102       Sai   15    Male        CSE    8.6          94        Yes
2         103       Jay   28    Male     CSE AI    8.9          93        Yes
3         104    Snehal   12  Female        CSE    9.3          92        Yes
4         105     Rahul   20    Male         IT    8.2          92         No
5         106     Pooja   18  Female     CSE AI    9.0          98        Yes
6         107     Kunal   19    Male         IT    8.4          90         No
7         108    Anjali   20  Female        CSE    9.2          91        Yes
8         109   Saurabh   19    Male     CSE AI    8.7          93        Yes
9         110      Neha   18  Female         IT    9.1          95        Yes
```

Read data from different fromats (CSV, XLS)

```
from google.colab import files
uploaded = files.upload()
print(uploaded)
```

Choose Files student_data.csv

student_data.csv(text/csv) - 41983 bytes, last modified: 1/24/2026 - 100% done

```
Saving student_data.csv to student_data (2).csv
{'student_data (2).csv': b'school,sex,age,address,famsize,Pstatus,Medu,Fedu,Mjob,Fjob,reason,guardian,traveltime,studytime,f
```

```
dff=pd.read_csv('student_data.csv')
dff
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goout	Dalc	Walc	hea
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4	3	4	1	1	
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5	3	3	1	1	
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4	3	2	2	3	
3	GP	F	15	U	GT3	T	4	2	health	services	...	3	2	2	1	1	
4	GP	F	16	U	GT3	T	3	3	other	other	...	4	3	2	1	2	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
390	MS	M	20	U	LE3	A	2	2	services	services	...	5	5	4	4	5	
391	MS	M	17	U	LE3	T	3	1	services	services	...	2	4	5	3	4	
392	MS	M	21	R	GT3	T	1	1	other	other	...	5	5	3	3	3	
393	MS	M	18	R	LE3	T	3	2	services	other	...	4	4	1	3	4	
394	MS	M	19	U	LE3	T	1	1	other	at_home	...	3	2	3	3	3	

395 rows × 33 columns

Find shape of data gives rows and column

```
df.shape
```

```
(10, 8)
```

Find missing value

```
df.isnull().sum()
```

	0
Student ID	0
Name	0
Age	0
Gender	0
Department	0
Marks	0
Attendance	0
Internship	0

dtype: int64

```
df.isnull().any()
```

	0
Student ID	False
Name	False
Age	False
Gender	False
Department	False
Marks	False
Attendance	False
Internship	False

dtype: bool

Finding data types

```
df.dtypes
```

	0
Student ID	int64
Name	object
Age	int64
Gender	object
Department	object
Marks	float64
Attendance	int64
Internship	object

dtype: object

**Finding out Zeros **

```
(df==0).sum()
```

	0
Student ID	0
Name	0
Age	0
Gender	0
Department	0
Marks	0
Attendance	0
Internship	0

dtype: int64

```
(df.select_dtypes(include=np.number)==0).sum()
```

	0
Student ID	0
Age	0
Marks	0
Attendance	0

dtype: int64

**Indexing ,selecting data and sorting **

```
df['Student ID']
```

	Student ID
0	101
1	102
2	103
3	104
4	105
5	106
6	107
7	108
8	109
9	110

dtype: int64

```
dff[['school']]
```

	school
0	GP
1	GP
2	GP
3	GP
4	GP
...	...
390	MS
391	MS
392	MS
393	MS
394	MS

395 rows × 1 columns

```
dff.iloc[0:5]
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goout	Dalc	Walc	health
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4	3	4	1	1	3
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5	3	3	1	1	3
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4	3	2	2	3	3
3	GP	F	15	U	GT3	T	4	2	health	services	...	3	2	2	1	1	5
4	GP	F	16	U	GT3	T	3	3	other	other	...	4	3	2	1	2	5

5 rows × 33 columns

```
dff[dff['age']>15]
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goout	Dalc	Walc	hea
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4	3	4	1	1	
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5	3	3	1	1	
4	GP	F	16	U	GT3	T	3	3	other	other	...	4	3	2	1	2	
5	GP	M	16	U	LE3	T	4	3	services	other	...	5	4	2	1	2	
6	GP	M	16	U	LE3	T	2	2	other	other	...	4	4	4	1	1	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
390	MS	M	20	U	LE3	A	2	2	services	services	...	5	5	4	4	5	
391	MS	M	17	U	LE3	T	3	1	services	services	...	2	4	5	3	4	
392	MS	M	21	R	GT3	T	1	1	other	other	...	5	5	3	3	3	
393	MS	M	18	R	LE3	T	3	2	services	other	...	4	4	1	3	4	
394	MS	M	19	U	LE3	T	1	1	other	at_home	...	3	2	3	3	3	

313 rows × 33 columns

Introduction and 10 minute

Sort Data

```
dff.sort_values(by='age',ascending=True)
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goout	Dalc	Walc	heal
21	GP	M	15	U	GT3	T	4	4	health	health	...	5	4	2	1	1	
20	GP	M	15	U	GT3	T	4	3	teacher	other	...	4	4	1	1	1	
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4	3	2	2	3	
3	GP	F	15	U	GT3	T	4	2	health	services	...	3	2	2	1	1	
36	GP	M	15	U	LE3	T	4	3	teacher	services	...	5	4	3	1	1	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
376	MS	F	20	U	GT3	T	4	2	health	other	...	5	4	3	1	1	
306	GP	M	20	U	GT3	A	3	2	services	other	...	5	5	3	1	1	
390	MS	M	20	U	LE3	A	2	2	services	services	...	5	5	4	4	5	
392	MS	M	21	R	GT3	T	1	1	other	other	...	5	5	3	3	3	
247	GP	M	22	U	GT3	T	3	1	services	services	...	5	4	5	5	5	

395 rows × 33 columns

Descibe Attributes & check Data types

```
dff.describe()
```

	age	Medu	Fedu	traveltime	studytime	failures	famrel	freetime	goout	Dalc	
count	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	3
mean	16.696203	2.749367	2.521519	1.448101	2.035443	0.334177	3.944304	3.235443	3.108861	1.481013	
std	1.276043	1.094735	1.088201	0.697505	0.839240	0.743651	0.896659	0.998862	1.113278	0.890741	
min	15.000000	0.000000	0.000000	1.000000	1.000000	0.000000	1.000000	1.000000	1.000000	1.000000	
25%	16.000000	2.000000	2.000000	1.000000	1.000000	0.000000	4.000000	3.000000	2.000000	1.000000	
50%	17.000000	3.000000	2.000000	1.000000	2.000000	0.000000	4.000000	3.000000	3.000000	1.000000	
75%	18.000000	4.000000	3.000000	2.000000	2.000000	0.000000	5.000000	4.000000	4.000000	2.000000	
max	22.000000	4.000000	4.000000	4.000000	4.000000	3.000000	5.000000	5.000000	5.000000	5.000000	

```
df.describe(include='all')
```

	Student ID	Name	Age	Gender	Department	Marks	Attendance	Internship
count	10.00000	10	10.000000	10	10	10.000000	10.000000	10
unique	NaN	10	NaN	2	3	NaN	NaN	2
top	NaN	Shantanu	NaN	Male	CSE AI	NaN	NaN	Yes
freq	NaN	1	NaN	6	4	NaN	NaN	8
mean	105.50000	NaN	18.700000	NaN	NaN	8.850000	93.300000	NaN
std	3.02765	NaN	4.083844	NaN	NaN	0.362859	2.311805	NaN
min	101.00000	NaN	12.000000	NaN	NaN	8.200000	90.000000	NaN
25%	103.25000	NaN	18.000000	NaN	NaN	8.625000	92.000000	NaN
50%	105.50000	NaN	18.500000	NaN	NaN	8.950000	93.000000	NaN
75%	107.75000	NaN	19.750000	NaN	NaN	9.100000	94.750000	NaN
max	110.00000	NaN	28.000000	NaN	NaN	9.300000	98.000000	NaN

Check data types again

```
dff.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   school      395 non-null    object
 1   sex         395 non-null    object
 2   age         395 non-null    int64
 3   address     395 non-null    object
 4   famsize     395 non-null    object
 5   Pstatus     395 non-null    object
 6   Medu        395 non-null    int64
 7   Fedu        395 non-null    int64
 8   Mjob        395 non-null    object
 9   Fjob        395 non-null    object
 10  reason      395 non-null    object
 11  guardian    395 non-null    object
 12  traveltime  395 non-null    int64
 13  studytime   395 non-null    int64
 14  failures    395 non-null    int64
 15  schoolsup   395 non-null    object
 16  famsup      395 non-null    object
 17  paid        395 non-null    object
 18  activities  395 non-null    object
 19  nursery     395 non-null    object
 20  higher      395 non-null    object
 21  internet    395 non-null    object
 22  romantic    395 non-null    object
 23  famrel      395 non-null    int64
 24  freetime    395 non-null    int64
 25  goout       395 non-null    int64
 26  Dalc        395 non-null    int64
 27  Walc        395 non-null    int64
 28  health      395 non-null    int64
 29  absences    395 non-null    int64
 30  G1          395 non-null    int64
 31  G2          395 non-null    int64
 32  G3          395 non-null    int64
dtypes: int64(16), object(17)
memory usage: 102.0+ KB
```

**Count unique value , format & Data types Conversion **

```
df.nunique()
```

	0
Student ID	10
Name	10
Age	6
Gender	2
Department	3
Marks	9
Attendance	7
Internship	2

dtype: int64

```
dff['age'].value_counts()
```

age	count
16	104
17	98
18	82
15	82
19	24
20	3
22	1
21	1

dtype: int64

```
dff.applymap(type)
```

```
/tmp/ipython-input-255471094.py:1: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.
  dff.applymap(type)
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goout	Dalc	Wal
0	<class 'str'>	<class 'str'>	<class 'int'>	<class 'str'>	<class 'str'>	<class 'str'>	<class 'int'>	<class 'int'>	<class 'str'>	<class 'str'>	...	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>
1	<class 'str'>	<class 'str'>	<class 'int'>	<class 'str'>	<class 'str'>	<class 'str'>	<class 'int'>	<class 'int'>	<class 'str'>	<class 'str'>	...	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>
2	<class 'str'>	<class 'str'>	<class 'int'>	<class 'str'>	<class 'str'>	<class 'str'>	<class 'int'>	<class 'int'>	<class 'str'>	<class 'str'>	...	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>
3	<class 'str'>	<class 'str'>	<class 'int'>	<class 'str'>	<class 'str'>	<class 'str'>	<class 'int'>	<class 'int'>	<class 'str'>	<class 'str'>	...	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>
4	<class 'str'>	<class 'str'>	<class 'int'>	<class 'str'>	<class 'str'>	<class 'str'>	<class 'int'>	<class 'int'>	<class 'str'>	<class 'str'>	...	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	..
390	<class 'str'>	<class 'str'>	<class 'int'>	<class 'str'>	<class 'str'>	<class 'str'>	<class 'int'>	<class 'int'>	<class 'str'>	<class 'str'>	...	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>
391	<class 'str'>	<class 'str'>	<class 'int'>	<class 'str'>	<class 'str'>	<class 'str'>	<class 'int'>	<class 'int'>	<class 'str'>	<class 'str'>	...	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>
392	<class 'str'>	<class 'str'>	<class 'int'>	<class 'str'>	<class 'str'>	<class 'str'>	<class 'int'>	<class 'int'>	<class 'str'>	<class 'str'>	...	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>
393	<class 'str'>	<class 'str'>	<class 'int'>	<class 'str'>	<class 'str'>	<class 'str'>	<class 'int'>	<class 'int'>	<class 'str'>	<class 'str'>	...	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>
394	<class 'str'>	<class 'str'>	<class 'int'>	<class 'str'>	<class 'str'>	<class 'str'>	<class 'int'>	<class 'int'>	<class 'str'>	<class 'str'>	...	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>	<class 'int'>

395 rows × 33 columns

**Converting variable data types **

```
dff['age']=dff['age'].astype(float)
```

```
dff['age']=dff['age'].astype(int)
```

```
dff['school']=dff['school'].astype('category')
```

Import as follows:

```
import numpy as np
import pandas as pd
```

Object Creation

See the Intro to data structures section. Creating a Series by passing a list of values, letting pandas create a default integer index:

```
s = pd.Series([1, 3, 5, np.nan, 6, 8])
s
```

	0
0	1.0
1	3.0
2	5.0
3	NaN
4	6.0
5	8.0

dtype: float64

Creating a DataFrame by passing a NumPy array, with a datetime index using date_range() and labeled columns:

```
dates = pd.date_range("20130101", periods=6)
dates
```

```
DatetimeIndex(['2013-01-01', '2013-01-02', '2013-01-03', '2013-01-04',
               '2013-01-05', '2013-01-06'],
              dtype='datetime64[ns]', freq='D')
```

```
df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list("ABCD"))
df
```

	A	B	C	D
2013-01-01	0.631567	-0.703992	1.221846	-1.062098
2013-01-02	-1.023470	-0.171616	-0.521982	-1.247480
2013-01-03	-0.431877	-0.468739	1.359356	1.159070
2013-01-04	0.199685	0.588314	-1.563385	-0.442762
2013-01-05	-0.666667	0.281978	-0.712405	-0.779234
2013-01-06	0.395033	-0.735824	-0.691703	-1.460358

Next steps: Generate code with df New interactive sheet

Creating a DataFrame by passing a dictionary of objects that can be converted into a series-like structure:

```
df2 = pd.DataFrame(
{
"A": 1.0,
"B": pd.Timestamp("20130102"),
"C": pd.Series(1, index=list(range(4)), dtype="float32"),
```

```
"D": np.array([3] * 4, dtype="int32"),
"E": pd.Categorical(["test", "train", "test", "train"]),
"F": "foo",
}
)
df2
```

	A	B	C	D	E	F
0	1.0	2013-01-02	1.0	3	test	foo
1	1.0	2013-01-02	1.0	3	train	foo
2	1.0	2013-01-02	1.0	3	test	foo
3	1.0	2013-01-02	1.0	3	train	foo

Next steps: Generate code with df2 New interactive sheet

The columns of the resulting DataFrame have different dtypes :

```
df2.dtypes
```

	0
A	float64
B	datetime64[s]
C	float32
D	int32
E	category
F	object

dtype: object

Viewing data

See the Basics section.

Use DataFrame.head() and DataFrame.tail() to view the top and bottom rows of the frame respectively:

```
df.head()
```

	A	B	C	D
2013-01-01	0.631567	-0.703992	1.221846	-1.062098
2013-01-02	-1.023470	-0.171616	-0.521982	-1.247480
2013-01-03	-0.431877	-0.468739	1.359356	1.159070
2013-01-04	0.199685	0.588314	-1.563385	-0.442762
2013-01-05	-0.666667	0.281978	-0.712405	-0.779234

Next steps: Generate code with df New interactive sheet

```
df.tail(3)
```

	A	B	C	D
2013-01-04	0.199685	0.588314	-1.563385	-0.442762
2013-01-05	-0.666667	0.281978	-0.712405	-0.779234
2013-01-06	0.395033	-0.735824	-0.691703	-1.460358

Display the DataFrame.index or DataFrame.columns :

```
df.index
```

```
DatetimeIndex(['2013-01-01', '2013-01-02', '2013-01-03', '2013-01-04',
               '2013-01-05', '2013-01-06'],
              dtype='datetime64[ns]', freq='D')
```

```
df.columns
```

```
Index(['A', 'B', 'C', 'D'], dtype='object')
```

DataFrame.to_numpy() gives a NumPy representation of the underlying data. Note that this can be an expensive operation when your DataFrame has columns with different data types, which comes down to a fundamental difference between pandas and NumPy: NumPy arrays have one dtype for the entire array, while pandas DataFrames have one dtype per column. When you call DataFrame.to_numpy() , pandas will find the NumPy dtype that can hold all of the dtypes in the DataFrame. This may end up being object , which requires casting every value to a Python object. For df , our DataFrame of all floating-point values, and DataFrame.to_numpy() is fast and doesn't require copying data:

```
df.to_numpy()
```

```
array([[ 0.6315673 , -0.70399225,  1.22184566, -1.06209791],
       [-1.02347001, -0.17161556, -0.52198223, -1.24748024],
       [-0.43187709, -0.46873852,  1.35935557,  1.15907038],
       [ 0.19968519,  0.58831352, -1.56338514, -0.44276209],
       [-0.66666668,  0.28197822, -0.71240529, -0.7792336 ],
       [ 0.39503314, -0.73582405, -0.69170341, -1.46035752]])
```

For df2 , the DataFrame with multiple dtypes, DataFrame.to_numpy() is relatively expensive:

```
df2.to_numpy()
```

```
array([[1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'test', 'foo'],
       [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'train', 'foo'],
       [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'test', 'foo'],
       [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'train', 'foo']],
      dtype=object)
```

NOTE: DataFrame.to_numpy() does not include the index or column labels in the output describe() shows a

quick statistic summary of your data:

```
df.describe()
```

	A	B	C	D
count	6.000000	6.000000	6.000000	6.000000
mean	-0.149288	-0.201646	-0.151379	-0.638810
std	0.654146	0.541824	1.175234	0.950276
min	-1.023470	-0.735824	-1.563385	-1.460358
25%	-0.607969	-0.645179	-0.707230	-1.201135
50%	-0.116096	-0.320177	-0.606843	-0.920666
75%	0.346196	0.168580	0.785889	-0.526880
max	0.631567	0.588314	1.359356	1.159070

Transposing your data:

```
df.T
```

	2013-01-01	2013-01-02	2013-01-03	2013-01-04	2013-01-05	2013-01-06
A	0.631567	-1.023470	-0.431877	0.199685	-0.666667	0.395033
B	-0.703992	-0.171616	-0.468739	0.588314	0.281978	-0.735824
C	1.221846	-0.521982	1.359356	-1.563385	-0.712405	-0.691703
D	-1.062098	-1.247480	1.159070	-0.442762	-0.779234	-1.460358

```
df.sort_values(by='B')
```

	A	B	C	D
2013-01-06	0.395033	-0.735824	-0.691703	-1.460358
2013-01-01	0.631567	-0.703992	1.221846	-1.062098
2013-01-03	-0.431877	-0.468739	1.359356	1.159070
2013-01-02	-1.023470	-0.171616	-0.521982	-1.247480
2013-01-05	-0.666667	0.281978	-0.712405	-0.779234
2013-01-04	0.199685	0.588314	-1.563385	-0.442762

Getting

Selecting a single column, which yields a Series , equivalent to df.A :

```
df["A"]
```

	A
2013-01-01	0.631567
2013-01-02	-1.023470
2013-01-03	-0.431877
2013-01-04	0.199685
2013-01-05	-0.666667
2013-01-06	0.395033

dtype: float64

Selecting via [] (**getitem**), which slices the rows:

```
df[0:3]
```

	A	B	C	D
2013-01-01	0.631567	-0.703992	1.221846	-1.062098
2013-01-02	-1.023470	-0.171616	-0.521982	-1.247480
2013-01-03	-0.431877	-0.468739	1.359356	1.159070

```
df["20130102":"20130104"]
```

	A	B	C	D
2013-01-02	-1.023470	-0.171616	-0.521982	-1.247480
2013-01-03	-0.431877	-0.468739	1.359356	1.159070
2013-01-04	0.199685	0.588314	-1.563385	-0.442762

Selection by label

Selecting on a multi-axis by label:

```
df.loc[:, ["A", "B"]]
```

	A	B
2013-01-01	0.631567	-0.703992
2013-01-02	-1.023470	-0.171616
2013-01-03	-0.431877	-0.468739

Showing label slicing, both endpoints are included:

```
df.loc["20130102":"20130104", ["A", "B"]]
```

	A	B
2013-01-02	-1.023470	-0.171616
2013-01-03	-0.431877	-0.468739
2013-01-04	0.199685	0.588314

Selection by position

See more in Selection by Position using DataFrame.iloc() or DataFrame.at() . Select via the position of the passed integers:

```
df.iloc[3]
```

	2013-01-04
A	0.199685
B	0.588314
C	-1.563385
D	-0.442762

dtype: float64

By integer slices, acting similar to NumPy/Python:

```
df.iloc[3:5, 0:2]
```

	A	B
2013-01-04	0.199685	0.588314
2013-01-05	-0.666667	0.281978

For slicing rows explicitly:

```
df.iloc[1:3, :]
```

	A	B	C	D
2013-01-02	-1.023470	-0.171616	-0.521982	-1.24748
2013-01-03	-0.431877	-0.468739	1.359356	1.15907

For slicing columns explicitly:

```
df.iloc[:, 1:3]
```

	B	C
2013-01-01	-0.703992	1.221846